Operators - Control Structure

By Sujal Shrestha

Table of

Contents

Operators

Python operators are special symbols or keywords used to perform operations on values or variables. They are categorized into several types:

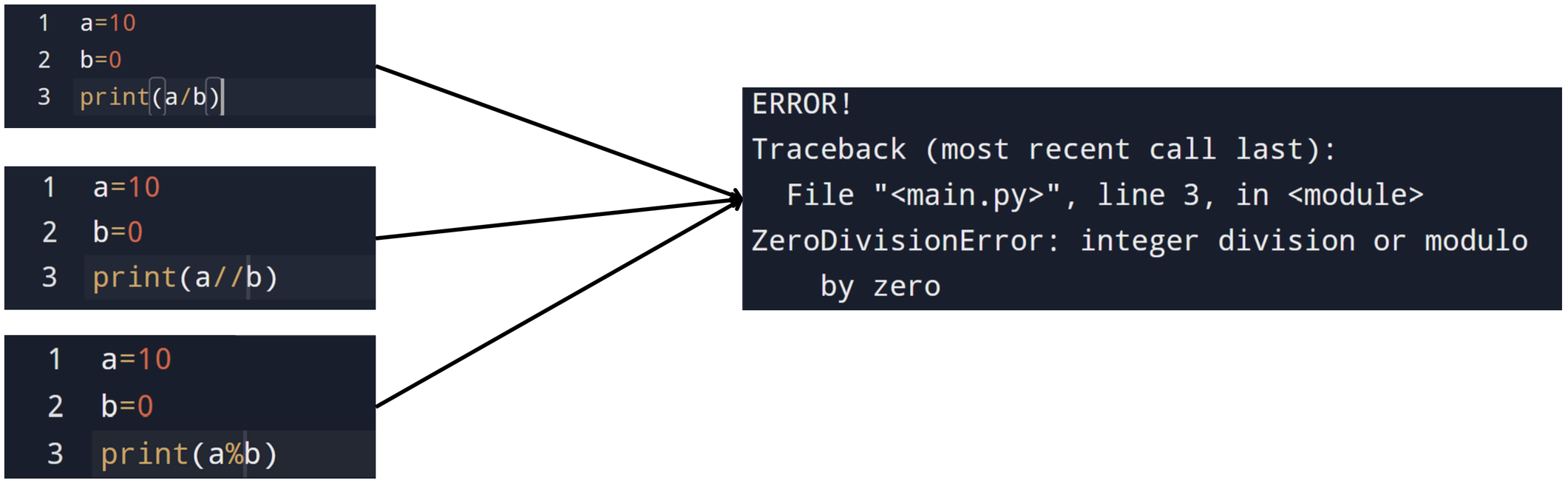
1. Arithmetic Operators
2. Relational or Comparison Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operator
6. Membership Operators
7. Identity Operators
8. Special Operators

Arithmetic Operator

Operators	Meaning	Example	Result
+	Addition	4 + 2	6
–	Subtraction	4 – 2	2
*	Multiplication	4 * 2	8
/	Division	4 / 2	2
%	Modulus operator to get remainder in integer division	5 % 2	1
**	Exponent	5**2 = 5^2	25
//	Integer Division/ Floor Division	5//2 -5//2	2 -3

Zero Division Error

In Python, a ZeroDivisionError occurs when a number is divided by zero using the /, //, or % operator.



Relational Operator

Relational Operators can be used to determine relative order. It is used to compare the operands, and return a Boolean result (True of False).

Operator	Meaning
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
! =	Not equal to

Relational Operator

Using Relational Operators with Number Types

Relational operators can compare numeric values such as integers and floats.

Example: x>y, x<=y

```
1  a=10
2  b=15
3  print(a>b)
```

Output

```
False
```

Relational Operator

Using Relational Operators with Character Types

Characters are compared based on their Unicode or ASCII values. We can get the ASCII value of any character using ord() function.

Example: ‘a’>’b’

```
1  c='A'
2  d='B'
3  print(c<d)
```

Output

```
True
```

Relational Operator

Using Relational Operators with Boolean Types

Boolean values (True and False) can also be compared. In Python, True is equivalent to 1 and False to 0.

Example: True>False.

```
1  c=True
2  d=False
3  print(c==d)
```

Output

```
False
```

Relational Operator

Chaining of Relational Operators

In the chaining, if all comparisons returns True then only result is True. If atleast one comparison returns False then the result is False.

Example: x < y <= z checks if x is less than y and y is less than or equal to z in one step.

```
1  a=10
2  b=15
3  c=20
4  print(a<b<c)
5  print(a<c<b)
```

Output

```
True
False
```

Equality Operator

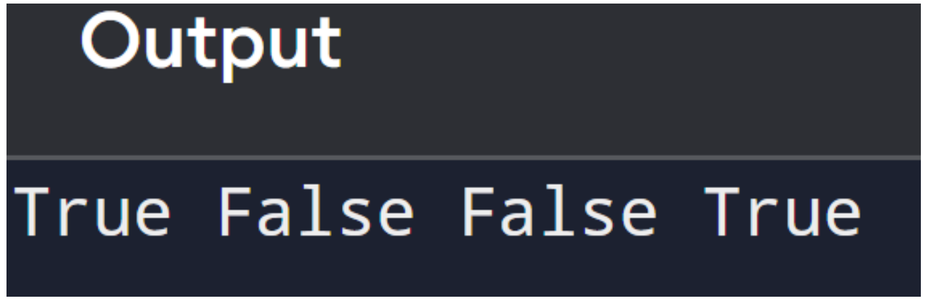
Equality operators are used to check whether the given two values are equal or not. The following are the equality operators used in Python.

1. Equal to (==)
2. Not Equal to (!=)

Example, 1 == 1, 1 == 1.0, 1 == True

Note: Chaining concept is applicable for equality operators.

```
1  a="LBEF"
2  b="LBEF"
3  c="lbef"
4  print(a==b, b==c, a!=b, b!=c)
```



Logical Operator

Logical Operator work with Boolean values (True or False) and are essential for decision-making in programming.

Following are the various logical operators used in Python.

1. and
2. or
3. not

Logical Operator

and operator

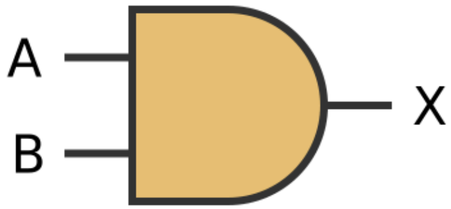
If both arguments are True then only result is True.

Syntax:

condition1 and condition2

```
1  print(True and True)
2  print(True and False)
3  print(False and True)
4  print(False and False)
```

```
True
False
False
False
```



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Logical Operator

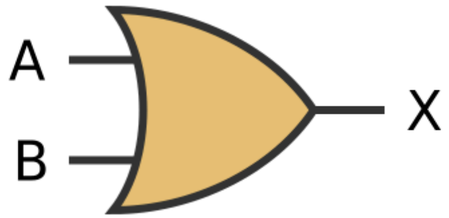
or operator

If atleast one arugemnt is True then result is True.

Syntax:

condition1 or condition2

```
1  print(True or True)                    True
2  print(True or False)                   True
3  print(False or True)                   True
4  print(False or False)                  False
```



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Logical Operator

not operator

Reverses the Boolean value of the condition.

- True becomes False.
- False becomes True.

The truth table for the 'not' operator

P	¬*P*
T	F
F	T

```
1  print(not True)
2  print(not False)
```

```
False
True
```

Assignment Operator “=”

The assignment operator in Python is used to assign a value to a variable.
Example, name =”Panacea Solutions”
Python provides serveral compound assignment coperators to perform operators and assign results simultaneously.

Syntax:
variable = value

Assignment Operator “=”

Operator	Purpose	Example	Equivalent
+=	Addition	x += 2	x = x + 2
-=	Subtraction	x -= 2	x = x - 2
/=	Division	x /= 2	x = x / 2
*=	Multiplication	x *= 2	x = x * 2
%=	Modulus	x %= 2	x = x % 2

Identity Operator

Identity operator in Python is used to compare the memory locations of two objects. They check whether two variables share the same memory address.

Types:

a. is

It reutrns true if both variables refers to the same object.

```
1  x="Team LBEF"
2  y=x
3  z="Team LBEF"
4  print( x is y, x is z)
```

```
True True

=== Code Execution Successful ===
```

Identity Operator

b. is not:

It returns True if both variables do not refer to the same object.

Example:

```
1  x=10
2  y=x
3  print( x is not y)
```

```
False

=== Code Execution Successful ===
```

```
1  x=10
2  y=35
3  print( x is not y)
```

```
True

=== Code Execution Successful ===
```

Difference Between is and ==

- is
  It checks whether two variables point to the same memory address.
- ==
  It checks whether the value of two variables are equal, regardless of whether they are the same object.

Example:

```
1  x=[10, 3.14, True, "LBEF"]
2  y=[10, 3.14, True, "LBEF"]
3  print( x is y )
4  print( x == y )
```

```
False
True


=== Code Execution Successful ===
```

Membership Operator

Membership operators (in) are used to test whether a value as a member of a sequence of collection, such as string, list, tuple, set, or dictionary.

Types of Membership Operators

1. in

   It returns True if the value is found in the sequence.

```
1  fruit="Apple"
2  print("p" in fruit)
3  print("b" in fruit)
```

```
True
False


=== Code Execution Successful ===
```

Membership Operator

2. not in

It returns True if value is not found in sequence.

Example:

```
1  fruit="Apple"
2  print("App" not in fruit)
3  print("b" not in fruit)
```

```
False
True


=== Code Execution Successful ===
```

Bitwise Operator

Bitwise operators are used in to perform bit-level operations. During computation, mathematical operations like: addition, subtraction, multiplication, division, etc are converted to bit-level which makes processing faster and saves power.

Bitwise Operator

Types of Bitwise Operators

Operator	Name	Example	Result
&	Bitwise AND	6 & 3	2
|	Bitwise OR	10 | 10	10
^	Bitwise XOR	2^2	0
~	Bitwise 1's complement	~9	-10
<<	Left-Shift	10<<2	40
>>	Right-Shift	10>>2	2

BITWISE OPERATORS

- Bitwise &, |, ^, >>, << are used for two operand data. But ~ is only used for single operand data.
- The result of the bitwise AND operation is 1 if both the bits have the value as 1; otherwise, the result is always 0.
- The result of the bitwise OR operation is 1 if at least one of the expression has the value as 1; otherwise, the result is always 0.
- The result of the bitwise Exclusive-OR operation is 1 if only one of the expression has the value as 1; otherwise, the result is always 0.

BITWISE OPERATORS

x	y	x&y	x|y	x^y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

BITWISE OPERATORS

- The bitwise complement Operator (~) is also called as one's complement operator since it always takes only one value or an operand. It is a unary operator. When we perform complement on any bits, all the 1's become 0's and vice versa.

  Eg:

  - int a=10, ~a=-11

  int a=-10, ~a=9

BITWISE OPERATORS

- The left shift operator (<<) is a type of Bitwise shift operator, which performs operations on the binary bits. It is a binary operator that requires two operands to shift or move the position of the bits to the left side and add zeroes to the empty space created at the right side after shifting the bits.

  Syntax:

  variable_name << no_of_position

  Example

  int a=20, a<<2 80

BITWISE OPERATORS

- The right shift operator (>>) is a type of bitwise shift operator used to move the bits at the right side, and it is represented as the double (>>) arrow symbol. Like the Left shift operator, the Right shift operator also requires two operands to shift the bits at the right side and then insert the zeroes at the empty space created at the left side after shifting the bits.

  Syntax:

  variable_name >> no_of_position

  Example

  int a=20, a>>2 □ 5

TERNARY OPERATORS

The ternary operator in Python lets you write an if-else statement in a single line.
It helps make the code shorter and cleaner for simple conditions.

Syntax:
value_if_true if condition else value_if_false

```
1  age = 18
2  print("Eligible to vote" if age>=18 else "Not eligible to
       vote")
```

```
Eligible to vote


=== Code Execution Successful ===
```

Operator Precedence

Operators	Meaning
`()`	Parentheses
`**`	Exponent
`+x`, `-x`, `~x`	Unary plus, Unary minus, Bitwise NOT
`*`, `/`, `//`, `%`	Multiplication, Division, Floor division, Modulus
`+`, `-`	Addition, Subtraction
`<<`, `>>`	Bitwise shift operators
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`==`, `!=`, `>`, `>=`, `<`, `<=`, `is`, `is not`, `in`, `not in`	Comparisons, Identity, Membership operators
`not`	Logical NOT
`and`	Logical AND
`or`	Logical OR

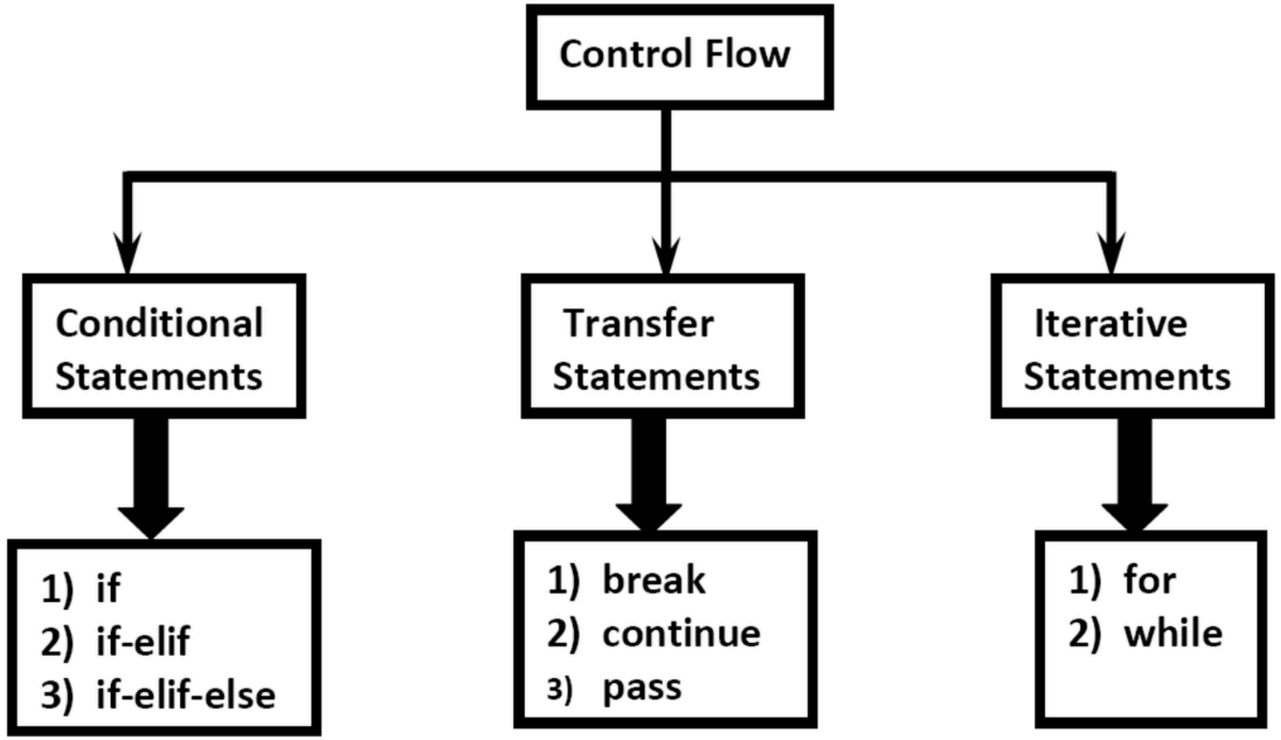
Control Structure

Flow Control Statements in Python

Flow control statements determine the order in which the instructions in a program are executed. They allow programs to make decisions, repeat actions, and handle specific conditions.

1. Conditional Statement
2. Iteration Statement (looping)
3. Transfer Statement/ Loop Control Statement

Flow Control Statements in Python



Conditional Statement

Conditional statements in Python allows to make decisions in program based on certain conditions. These statements execute a block of code only if the specified condition evaluates to True.

Note:
There is no switch statement in Python. (Which is available in C and Java).
There is no do-while loop in Python.(Which is available in C and Java).
 goto statement is also not available in Python. (Which is available in C).

Conditional Statement

Types of conditional statement:

1. if statement
2. if-else statement
3. if-elif-else statement

Conditional Statement

if statement:
The if statement is used for decision-making in Python. It allows you to execute a block of code only if a condition is true.

Syntax:

```
if condition:
    # code
```

Note:
In Python, a colon (:) is used to indicate the start of a block of code, such as if, elif, else, def, for, while, etc.

Conditional Statement

if statement:

Example:

Write a python programme to allow vote to candidate if his age is above 18.

```
1  age=18
2  if age>=18:
3      print("Eligible to vote.")
4
```

```
Eligible to vote.


=== Code Execution Successful
```

Note:

Here, Indentation Defines the Scope of the Block. If you are not followed indentation, you will get indentation error.

Conditional Statement

if-else statement:
The if-else statement is a conditional control structure used to execute one block of code if a condition is true, and a different block if the condition is false.

Syntax:

```
if condition:
   #Action 1
else:
    #Action 2
```

Conditional Statement

if-else statement:

Example:

Write a python programme to find if its raining or not.

```
is_raining = False
if is_raining:
    print("Take an umbrella.")
else:
    print("Not raining.")

```

```
Not raining.

=== Code Execution Successful ===
```

Conditional Statement

if-elif-else statement:

The if-elif-else structure is used to check multiple conditions. Python checks each condition from top to bottom, and executes the block of the first true condition.

Syntax:

```
if condition1:
   Action-1
elif condition2:
   Action-2
else:
 Default Action
```

Conditional Statement

if-elif-else statement:

Example: Write a python programme to check temperature and display.

temperature>30 -“Hot”, temperature>20 -“Warm”, temperature>10 - “Cool”

else -“Cold”

```
temperature = 15

if temperature > 30:
    print("It's hot.")
elif temperature > 20:
    print("It's warm.")
elif temperature > 10:
    print("It's cool.")
else:
    print("It's cold.")
```

```
It's cool.

=== Code Execution Successful ===
```

Conditional Statement

nested if-else statement:

A nested if-else is when an if or else block contains another if-else inside it. It is used when decisions depend on multiple conditions one inside another.

Syntax:

```
if condition1:
    if condition2:
        #Action
    else:
        #Action
else:
    #Default Action
```

Conditional Statement

nested if-else statement:

A nested if-else is when an if or else block contains another if-else inside it. It is used when decisions depend on multiple conditions one inside another.

Syntax:

```
if condition1:
    if condition2:
        #Action
    else:
        #Action
else:
    #Default Action
```

Conditional Statement

nested if-else statement:

A person is eligible for a car loan if:

- Age is 21 or older
  - If income is more than 25,000, they are eligible
  - Otherwise, not eligible due to low income
- If under 21, they are not eligible due to age

Conditional Statement

nested if-else statement:

```
1  age=int(input("Enter age: "))
2  income=float(input("Enter income: "))
3  if age>=21:
4      if income>=25000:
5          print(" Eligible for car loan.")
6      else:
7          print(" Not eligible due to low income.")
8  else:
9      print("No eligible, due to age.")
```

```
Enter age: 22
Enter income: 22000
 Not eligible due to low income.

=== Code Execution Successful ===
```

Conditional Statement

1. Write a Python program that takes two numbers as input and prints the largest number.
2. Write a Python program to input three numbers and print the smallest one using if, elif, and else.
3. Take an integer input from the user and print whether the number is positive, negative, or zero.
4. Write a program that checks if the number entered by the user is even or odd using conditional statements.
5. If a customer purchases goods worth more than Rs. 5000, give a 10% discount. Otherwise, charge full amount. Ask the user to enter the total bill and print the final payable amount.

Questions

Write a Python program that takes marks (0–100) and prints the grade:

A: 90–100

B: 80–89

C: 70–79

D: 60–69

F: below 60

Iterative Statement

In programming, iteration means repeating a block of code multiple times until a condition is met. Python provides statements that allow repetition, so we do not have to write the same code again and again.

Types of iterative statement in Python,

1. for loop
2. while loop

for loop

For is used when we already know how many times we want to repeat something, or when we want to go through every item in a sequence like a list, string, or a range of numbers.
You can think of a for loop as a tool that helps you visit each item one by one and do something with it.

Syntax:

```
for variable in sequence:
    statements
```

for loop

Breakdown of the syntax:

- for → tells Python we are starting a loop
- variable → a temporary name that represents each item in the sequence
- sequence → the collection we want to loop through (list, string, range, etc.)
- : → marks the beginning of the loop block
- Indented statements → the code that will run every time the loop repeats

FOR LOOP WITH STRING

Strings in Python are iterable, meaning you can loop through each character in a string using a for loop.

Syntax:

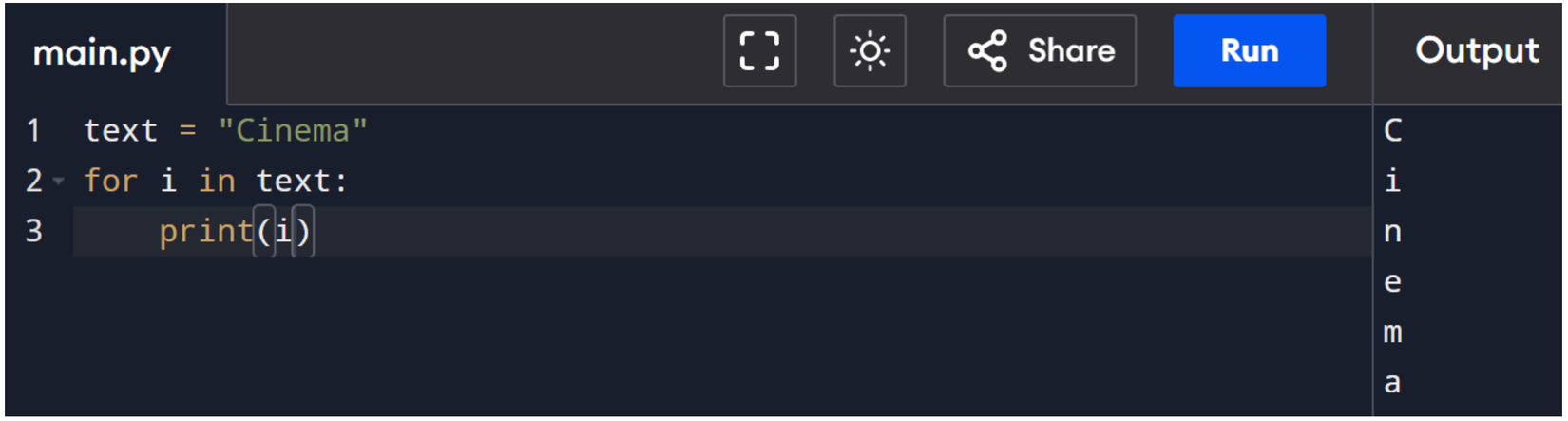
```
for char in string_variable:
    # Code to process each character
    print(char)
```

FOR LOOP WITH STRING

A for loop picks items from the sequence one at a time.
Each time:

1. It takes the next item
2. Stores it inside the loop variable
3. Runs the loop body using that value

This continues until the sequence has no more items left



FOR LOOP WITH LIST

main.py

Share Run

```
1  animal = ["Dog", "Cat", "Cow", "Goat"]
2  for i in animal:
3      print(i)
```

Output

```
Dog
Cat
Cow
Goat

=== Code Execution Successful ===
```

FOR LOOP WITH STRING

Strings in Python are iterable, meaning you can loop through each character in a string using a for loop.

Syntax:

```
for char in string_variable:
    # Code to process each character
    print(char)
```

FOR LOOP WITH RANGE()

Range() function in Python is used to repeat a block of code a specific number of times. The range() function generates a sequence of numbers, which the for loop iterates through.

Parameters of range():

1. start: The starting value of the sequence (default is 0).
2. stop: The ending value (not included in the sequence).
3. step: The difference between each number (default is 1).

FOR LOOP WITH RANGE()

Syntaxt:

```
for var in range(start, stop, step):
   #Code
```

Examples:

1. Using range(stop) (Default Start = 0)

```
for i in range(10):
```

2. Using range(start, stop)

```
 for i in range(1, 10):
```

3. Using range(start, stop, step)

```
 for i in range(1, 10, 2):
```

FOR LOOP WITH RANGE()

```
1 for i in range(1,10):
2     print(i)
```

```
1
2
3
4
5
6
7
8
9

=== Code Execution Successful ===
```

QUESTION

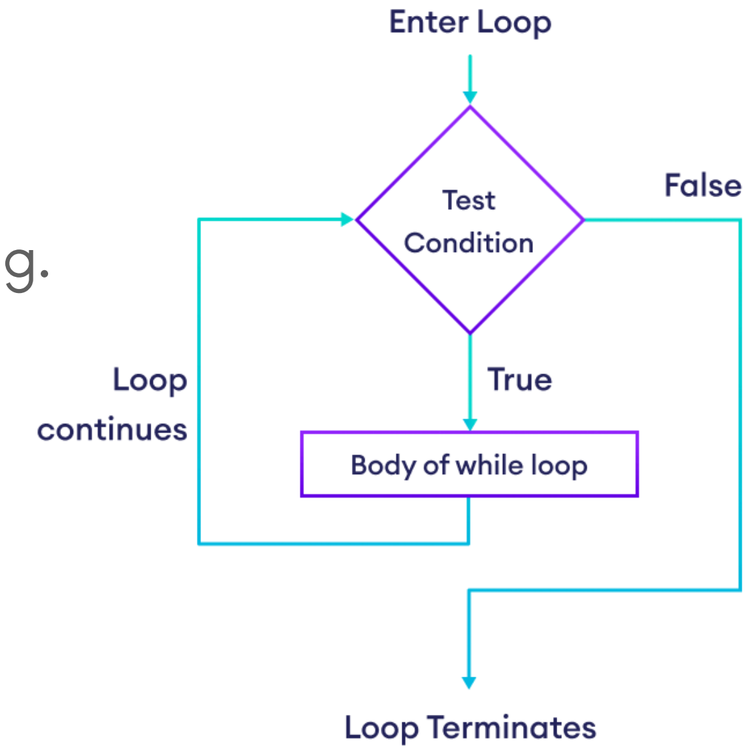
1. Print numbers from 1 to 10.
2. Print numbers in reverse from 10 to 1.
3. Print the multiplication table of 7.
4. Print the odd number between 1 -100.
5. Print the number divisible be 5 and 7.
6. Write a program to count the number of vowels in a given string using a for loop.
7. Given a list of numbers [12, 15, 56, 5 , 4, 97, 1, 17, 19], write a program to find the largest number using a for loop.
8. Write a program to calculate the sum of first 10 natural numbers using a for loop.
9. Write a program to calculate the factorial of a number using a for loop.
10. Write a program to print the Fibonacci series up to n terms using a for loop.

WHILE LOOP

A while loop is used to repeat a block of code as long as a condition is true. The loop continues running until the condition becomes false. We mainly use a while loop when:

1. The number of repetitions is not fixed
2. Execution depends on a condition

Example, While the battery is not 100%, keep charging.



WHILE LOOP

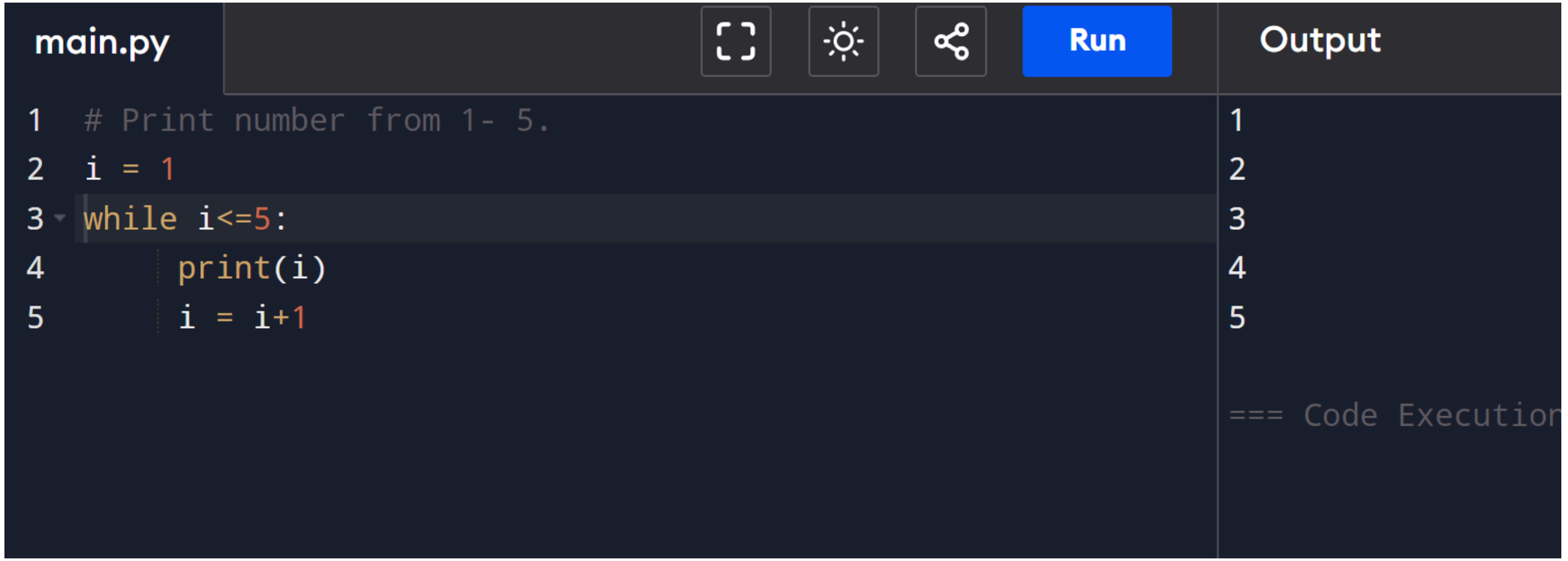
Syntax of while loop:

```
while condition:
    statement(s)
```

Meaning of Syntax

- while → keyword that starts the loop
- condition → a Boolean expression (True or False)
- :` → indicates the beginning of the loop body
- Indented statements → code that repeats while the condition is true

WHILE LOOP


main.py
Run
Output
1 # Print number from 1- 5.
2 i = 1
3 while i<=5:
4 print(i)
5 i = i+1
1
2
3
4
5
=== Code Execution

NESTED LOOPS

A nested loop means a loop inside another loop. The inner loop runs completely for every single iteration of the outer loop.

Syntax:

```
for i in range(outer_limit):
    for j in range(inner_limit):
        # statements
```

NESTED LOOPS

Example: Print the shape pattern square, take rows and coulmns from user.

```
1  rows = int(input("Enter rows: "))
2  columns = int(input("Enter columns: "))
3  for i in range(rows):
4      for j in range(columns):
5          print("*", end="\t")
6      print()
7
8
```

```
Enter rows: 5
Enter columns: 5
*	*	*	*	*
*	*	*	*	*
*	*	*	*	*
*	*	*	*	*
*	*	*	*	*

=== Code Execution Successful ===
```

LOOP CONTROL /TRANSFER STATEMENT

Loop control statements alter the normal flow of a loop — either by skipping, stopping, or doing nothing in a particular iteration.

1. break
2. continue
3. pass

LOOP CONTROL /TRANSFER STATEMENT

break:

break is used to terminate the loop immediately, even if the loop condition is still true.

Eg. break the loop after 5.

```
for i in range(1, 10):
    print(i)
    if i==5:
        print("Loop is terminated!")
        break
```

```
1
2
3
4
5
Loop is terminated!
```

LOOP CONTROL /TRANSFER STATEMENT

continue:

continue skips the current iteration and moves to the next one.

Eg. skip the value 3 in a loop.

```
for i in range(1, 10):
    if i==3:
        continue
    print(i)
print("3 is skipped!")
```

```
1
2
4
5
6
7
8
9
3 is skipped!
```

LOOP CONTROL /TRANSFER STATEMENT

pass

pass does nothing. It is used as a placeholder when the syntax requires a block.

Eg.

```
for i in range(1,10):
    pass

def func():
    pass

x=15
if x>10:
    pass
```

```
=== Code Execution Successful ===
```

WHILE TRUE

In Python, while True creates a loop that runs indefinitely (forever), and continues execution until it is explicitly stopped using **break.**

Syntax:

while True:

    statement(s)

```
1  # Write a python program to ask user to stop the program or not
       . Only stop the code when user type stop.
2
3  while True:
4      print("Loop Forever")
5      chk = input("Type Stop to exit: ").lower()
6      if chk=="stop":
7          print("Loop is stopped!")
8          break
```

```
Loop Forever
Type Stop to exit: a
Loop Forever
Type Stop to exit: a
Loop Forever
Type Stop to exit: s
Loop Forever
Type Stop to exit: st
Loop Forever
Type Stop to exit: stop
Loop is stopped!

=== Code Execution Successful ===
```

WHILE TRUE

Questions:
A mobile recharge system keeps showing options until the user chooses to exit.
Menu:

1. Check Balance => Display “Your balance is Rs. 150.00 “
2. Exit

Question:
Write a Python program using while True that displays the above menu repeatedly.
The program should stop only when the user selects Exit.

WHILE TRUE

Questions:

An ATM provides the following services:

1. Check Balance => Display “Your balance is Rs..... .”
2. Withdraw Money => Display “Rs..... is withdrawn.”
3. Deposit Money => Display “Rs..... is deposited.”
4. Exit

Question:

Write a Python program using while True that displays the ATM menu repeatedly. The program should terminate only when the user selects Exit.

WHILE TRUE

Questions:
A mobile recharge system keeps showing options until the user chooses to exit.
Menu:

1. Check Balance => Display “Your balance is Rs. 150.00 “
2. Exit

Question:
Write a Python program using while True that displays the above menu repeatedly.
The program should stop only when the user selects Exit.

ENUMERATE()

enumerate() is a built-in Python function used to get both the index (position) and the value of items while looping through a sequence (like a list, tuple, or string).

Syntax:

enumerate(iterable, start=0)

main.py

Share Run

```
1 fruits=["Apple", "Banana", "Cherry","Orange", "Kiwi"]
2 for index, fruit in enumerate(fruits,start=1):
3     print(index,fruit)
```

Output

```
1 Apple
2 Banana
3 Cherry
4 Orange
5 Kiwi

=== Code Execution Successful ===
```